

Laboratory 4 Report

ANDREA BOTTICELLA*, s347291, Politecnico di Torino, Italy

ELIA INNOCENTI*, s345388, Politecnico di Torino, Italy

SIMONE ROMANO*, s344024, Politecnico di Torino, Italy

This laboratory studies malicious SSH sessions as sequences of Bash entities and their associated MITRE ATT&CK tactics. We first characterize the labelled dataset and examine how tactic assignment depends on command context. We then compare a general-purpose BERT tokenizer with the code-oriented UniXcoder tokenizer, quantify their behavior on shell commands and complete sessions, and apply word truncation to control pathological token growth. Subsequent stages fine-tune and compare transformer models for tactic prediction and use the best configuration to analyse unlabelled honeypot sessions.

1 INTRODUCTION

SSH honeypots record commands, flags, parameters, and separators entered by remote users. In this laboratory, each space-separated Bash entity is treated as an SSH word and is associated with a MITRE tactic describing the adversary's intent. The objective is to build a Named Entity Recognition pipeline that can learn these contextual labels from a small annotated dataset and later support the analysis of unlabelled sessions.

The workflow comprises four stages: **dataset characterization**, **tokenization analysis**, **transformer fine-tuning**, and **inference with tactic fingerprints**. This report follows the same order. All payloads are treated strictly as text: long encoded strings are truncated when required and are never decoded or executed.

2 TASK 1: DATASET CHARACTERIZATION

The labelled data comprise **251 training sessions** and **108 test sessions**. Each session is represented by a raw command string and a list of MITRE tactics aligned with its space-separated Bash words.

2.1 Label Distribution

Distinct tactics and imbalance. The dataset contains seven tactics: Discovery, Execution, Persistence, Impact, Defense Evasion, Not Malicious Yet, and Other. Figure 1 reports their word-level frequencies in both partitions. The two distributions have a similar shape: Discovery dominates with **52.37%** of training words and **53.36%** of test words, followed by Execution (**28.23%/25.30%**) and Persistence (**9.87%/11.02%**). Every remaining class accounts for less than 4% of either split.

This imbalance is important for the modelling stage: accuracy alone will be insufficient, because a classifier can perform well on the dominant tactics while neglecting minority behaviors such as Impact or Other. Macro-averaged metrics and per-class F1 scores are therefore necessary.

2.2 Contextual Meaning of echo

Observed labels. In the training set, echo occurs with six different tactics: Persistence (104 occurrences), Execution (39), Discovery (31), Not Malicious Yet (8), Impact (6), and Other (4). A command name is therefore not sufficient to determine its tactic.

Persistence example. In the sequence `echo root:<password> | chpasswd | bash`, the emitted string is consumed by `chpasswd` to modify credentials. The command contributes to maintaining access and is consequently labelled Persistence.

Execution example. In `echo <encoded payload> | base64 -decode | bash`, the output is decoded and passed directly to a shell. Here, echo participates in launching a payload and is labelled Execution. These examples show that tactic assignment depends on neighboring words and downstream use, motivating a contextual sequence model rather than a command-to-label lookup table.

2.3 Session Lengths

Length variability. Training sessions range from 2 to 224 Bash words, with mean **45.70** and median **17**. Test sessions have the same range but are generally longer, with mean **57.38** and median **23.5**. The large gap between means and medians indicates a right-skewed distribution: many sessions are short, while a small number of long traces extend the tail.

Figure 2 visualizes the empirical cumulative distribution over the labelled sessions. The long tail is relevant because the number of Bash words does not directly equal the number of model tokens: unusual shell strings can be fragmented into many subwords and exceed a transformer's context limit even when the word count appears manageable.

*The authors collaborated closely in developing this project.

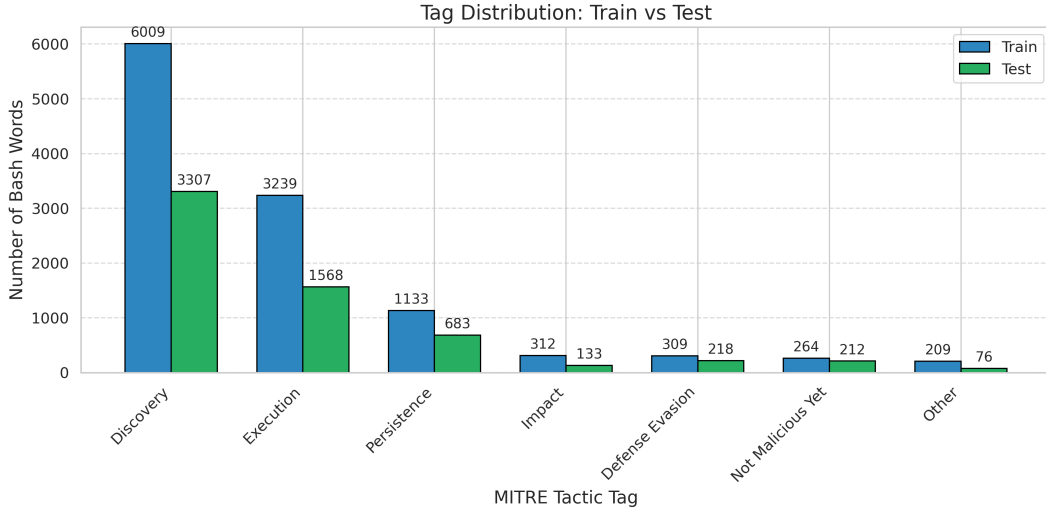


Fig. 1. Distribution of MITRE tactic labels over Bash words in the training and test partitions.

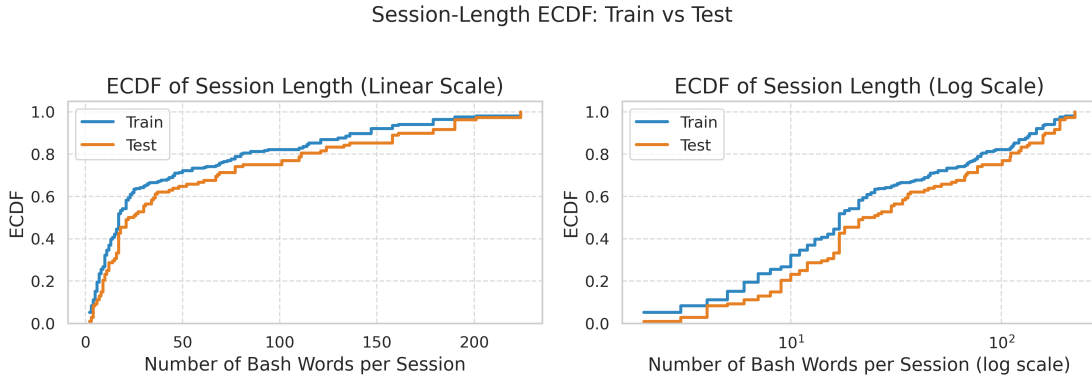


Fig. 2. ECDF of the number of Bash words per labelled SSH session in the training and test partitions, shown on linear and logarithmic scales.

3 TASK 2: TOKENIZATION

We compare **BERT-base uncased**, trained primarily on natural-language text, and **UniXcoder-base**, whose pre-training also includes source code. Both tokenizers are evaluated with a maximum sequence length of 512.

3.1 Tokenization of SSH Commands

Command-level comparison. Table 1 shows the subword decomposition of the nine required commands. Prefix markers used by the tokenizers are omitted from the table for readability. BERT uses ## for continuation pieces, whereas UniXcoder uses a spacing marker on word-initial tokens.

BERT produces 16 tokens for the nine words, a token-to-word ratio of **1.78**; UniXcoder produces 14, yielding the better ratio of **1.56**. Common words such as `cat`, `echo`, and `shell` are retained by both vocabularies because they are sufficiently frequent in their pre-training corpora. UniXcoder additionally preserves `grep`, consistent with its exposure to code and command-line text.

3.2 Full-Corpus Tokenization and Context Limit

Token counts before preprocessing. Table 2 summarizes tokenization of all 251 training sessions. Although UniXcoder has a lower median, its mean and maximum are much larger because rare shell strings are decomposed rather than collapsed into unknown tokens. Across the corpus, BERT emits **303 [UNK] tokens**, whereas UniXcoder emits none.

Consequently, **24 BERT sessions (9.56%)** and **29 UniXcoder sessions (11.55%)** exceed 512 tokens. Figure 3 shows the corresponding ECDFs and the context threshold. Most sessions remain below the limit, but UniXcoder has a visibly long tail.

Longest session. Both maxima occur on the same **134-word** training session. Two words exceed 30 characters; the longest is a **46,860-character encoded payload**, which is kept strictly as text. BERT replaces portions of the rare sequence with [UNK] and produces 1,889 tokens, whereas UniXcoder decomposes it into 28,920 tokens. The code-oriented tokenizer is therefore compact on ordinary commands but much more verbose on pathological strings.

Table 1. Subword tokenization of the selected SSH commands. Table 2. Training-corpus tokenization before word truncation.

Command	BERT	UniXcoder
cat	cat	cat
shell	shell	shell
echo	echo	echo
top	top	top
chpasswd	ch, pass, wd	ch, passwd
crontab	cr, ont, ab	c, ront, ab
wget	w, get	w, get
busybox	busy, box	busy, box
grep	gr, ep	grep

Metric	BERT	UniXcoder
Mean tokens/session	178.59	409.32
Median tokens/session	71	62
Maximum tokens/session	1,889	28,920
Corpus [UNK] count	303	0
Sessions above 512	24	29

Tokenization Distributions and Truncation Analysis

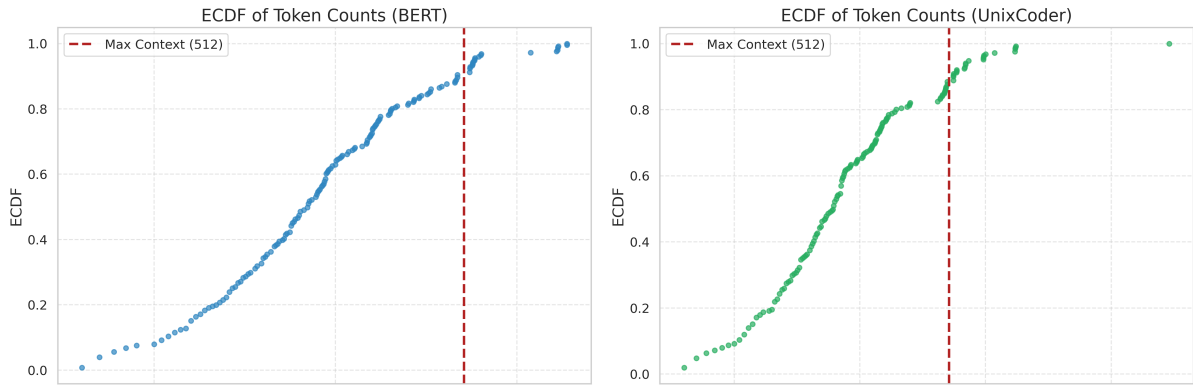


Fig. 3. ECDF of token counts before preprocessing. The dashed line marks the 512-token context limit.

Words vs Tokens (After Word Truncation)

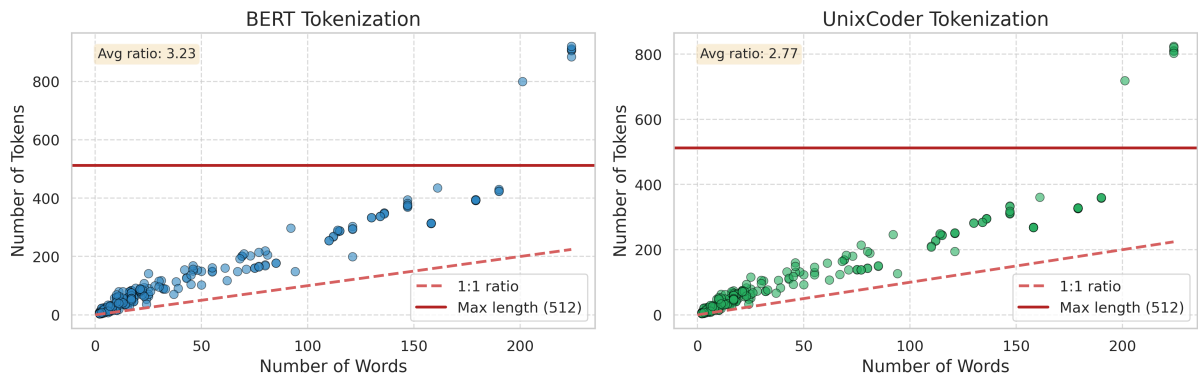


Fig. 4. Number of Bash words versus generated tokens after truncating individual words to 30 characters.

3.3 Truncating Long Bash Words

Preprocessing effect. Following the prescribed preprocessing, every Bash word longer than 30 characters is shortened before tokenization. This reduces the average sequence length to **128.38 tokens for BERT** and **110.52 for UniXcoder**; the maxima fall to 920 and 824, respectively. Only **six sessions per tokenizer** remain above the 512-token limit.

Figure 4 relates Bash-word counts to token counts after preprocessing. Using the notebook's per-session definition $\frac{1}{N} \sum_i \frac{\text{tokens}_i}{\text{words}_i}$, the average token-to-word ratio is **3.23 for BERT** and **2.77 for UniXcoder**. Thus, once abnormally long words are controlled, UniXcoder provides the more compact representation of typical SSH syntax.